Comparative Evaluation of ToT on Mathematical Reasoning: A Game of 24 Study

1st Lu Nguyen Hai Trieu
Faculty of Information Systems
University of Information Technology
Ho Chi Minh City, Vietnam
23521656@gm.uit.edu.vn

2nd Nguyen Van Nguyen
Faculty of Information Systems
University of Information Technology
Ho Chi Minh City, Vietnam
23521053@gm.uit.edu.vn

3rd Nguyen Duc Trung
Faculty of Information Systems
University of Information Technology
Ho Chi Minh City, Vietnam
23521677@gm.uit.edu.vn

4th Le Anh Tuan
Faculty of Information Systems
University of Information Technology
Ho Chi Minh City, Vietnam
23521710@gm.uit.edu.vn

***Abstract*—Large Language Models (LLMs) have demonstrated strong capabilities in natural language understanding and generation. However, they still struggle with complex multi-step reasoning tasks. Chain-of-Thought (CoT) prompting improves reasoning performance by encouraging intermediate reasoning steps, but it remains limited by its linear reasoning structure and lack of backtracking capability. Tree of Thoughts (ToT) has recently been proposed as a reasoning framework that enables LLMs to explore multiple reasoning paths through tree-based search strategies such as Breadth-First Search (BFS). This paper presents a comparative evaluation of ToT on mathematical reasoning using the Game of 24 benchmark. We reproduce the ToT framework using GPT-3.5 and compare its performance against Input-Output (IO) prompting and CoT prompting. In addition, we investigate the impact of beam width in BFS-based ToT on reasoning accuracy and token usage. Experimental results show that ToT with beam widths of 3 and 5 outperforms conventional prompting methods on the evaluated problems, and that increasing the beam width from 3 to 5 improves accuracy from 10% to 30% at the cost of a 1.4× increase in API cost. The findings of this study highlight the effectiveness of structured reasoning and search-based exploration in enhancing the problem-solving capabilities of LLMs. This work also provides an accessible reproduction and analysis framework for future research on LLMs reasoning strategies.**

***Index Terms*—Large language models, tree of thoughts, chain-of-thought prompting, mathematical reasoning, Game of 24, beam search**

I. Introduction

Large language models (LLMs) have recently achieved remarkable progress in natural language processing tasks, including text generation, question answering, summarization, and code generation. Models such as GPT [1], [2] have demonstrated strong capabilities in understanding contextual information and generating human-like responses. Despite these advances, LLMs still face significant challenges when solving complex reasoning tasks that require multi-step planning, intermediate decision making, and error correction. In particular, mathematical reasoning tasks often expose the limitations of conventional prompting approaches, where a single incorrect intermediate step may lead to an entirely incorrect final answer.

To improve reasoning performance, recent studies have introduced prompting techniques that encourage models to generate intermediate reasoning steps before producing a final answer. One widely adopted approach is chain-of-thought (CoT) prompting [3], which enables LLMs to decompose complex problems into sequential reasoning steps. Although CoT significantly improves reasoning accuracy compared to direct input-output (IO) prompting, its reasoning process remains fundamentally linear. Once the model commits to an incorrect reasoning path, it lacks an effective mechanism for revisiting or exploring alternative solutions.

To address these limitations, the Tree of Thoughts (ToT) framework [4] was proposed as a search-based reasoning strategy for LLMs. Instead of generating a single reasoning chain, ToT allows the model to explore multiple intermediate reasoning paths organized as a tree structure. By integrating search algorithms such as breadth-first search (BFS), the framework enables the model to evaluate candidate reasoning states, backtrack from unpromising branches, and search for more effective solution paths. This structured exploration process has shown promising results on tasks requiring planning and mathematical reasoning.

Motivated by the growing importance of reasoning capabilities in LLMs, this paper presents a comparative evaluation of the ToT framework on mathematical reasoning using the Game of 24 benchmark. We reproduce the original ToT framework using GPT-3.5 and compare its performance against conventional prompting methods, including IO prompting and CoT prompting. In addition, we investigate the effect of beam width ($b = 3$ and $b = 5$) in BFS-based ToT on reasoning accuracy and token consumption.

The main contributions of this work are threefold. First, we reproduce the ToT reasoning framework using GPT-3.5 on the Game of 24 benchmark. Second, we compare IO prompting, CoT prompting, and BFS-based ToT, and analyze the impact of beam width on reasoning performance and computational cost. Third, we analyze the trade-off between reasoning accuracy and computational overhead in search-based reasoning frameworks.

II. Related Research

A. LLMs and Reasoning

LLMs have significantly advanced the field of natural language processing by demonstrating strong performance across a wide range of tasks, including text generation, machine translation, summarization, and question answering. Transformer-based architectures, particularly the GPT series [1], [2], have shown emergent reasoning capabilities when scaled to large model sizes and trained on massive datasets. These models are capable of generating coherent and contextually relevant responses by leveraging self-attention mechanisms and large-scale pretraining.

Despite these advances, complex reasoning tasks remain challenging for LLMs. Tasks involving mathematical reasoning, planning, and multi-step problem solving often require models to maintain logical consistency across intermediate reasoning steps. In many cases, a single incorrect intermediate decision may lead to an incorrect final answer. Consequently, improving reasoning capabilities has become an important research direction for modern language models.

B. Prompt Engineering Techniques

Prompt engineering has emerged as an effective approach for improving the performance of LLMs without requiring additional model training. Instead of modifying model parameters, prompting methods guide the reasoning process by carefully designing input instructions and examples. Early prompting approaches mainly relied on direct IO prompting, where models generate answers immediately from user queries.

Although direct prompting performs well on simple tasks, it often struggles with tasks requiring structured reasoning and intermediate decision-making. To address these limitations, researchers introduced prompting techniques that encourage models to explicitly generate reasoning steps before producing final answers. These methods have shown substantial improvements in mathematical reasoning and logical inference tasks [3], [5].

C. CoT Reasoning

CoT prompting is one of the most influential reasoning techniques proposed for LLMs. The method improves reasoning performance by encouraging models to generate intermediate reasoning steps that simulate human problem-solving processes. Previous studies demonstrated that CoT prompting [3] significantly improves performance on arithmetic, commonsense, and symbolic reasoning tasks compared to standard prompting approaches.

Despite its effectiveness, CoT reasoning remains fundamentally linear. The model follows a single reasoning trajectory from beginning to end without revisiting earlier decisions or exploring alternative reasoning paths. As a result, incorrect intermediate reasoning steps can propagate throughout the reasoning chain and lead to incorrect final outputs. Furthermore, CoT lacks explicit mechanisms for evaluating intermediate states or performing systematic exploration during problem solving. These limitations motivate the development of more structured reasoning frameworks capable of exploring multiple candidate solutions.

D. Search-Based Reasoning in LLMs

Search-based reasoning methods integrate classical search algorithms with language model reasoning processes to improve decision-making and exploration capabilities. Instead of relying on a single reasoning path, these approaches allow language models to explore multiple candidate states before selecting an appropriate solution. Search strategies such as BFS have been widely studied in artificial intelligence for solving planning and combinatorial optimization problems.

In the context of LLMs, search-based reasoning enables models to generate multiple reasoning branches, evaluate intermediate thoughts, and backtrack from unpromising solution paths. This structured exploration mechanism can reduce the impact of early reasoning errors and improve robustness in complex reasoning tasks. However, integrating search strategies with LLM reasoning also introduces additional computational overhead, including increased token consumption.

E. ToT Framework

ToT [4] was recently proposed as a deliberate reasoning framework for LLMs. Unlike CoT prompting, which generates a single reasoning sequence, ToT organizes intermediate reasoning steps into a tree structure that supports exploration, evaluation, and backtracking. The framework allows models to generate multiple candidate thoughts at each reasoning step and evaluate their potential usefulness before continuing the search process. The original ToT framework demonstrated strong performance improvements on tasks requiring planning and mathematical reasoning, including the Game of 24 benchmark. By combining language model reasoning with search algorithms such as BFS, ToT provides a more flexible and structured reasoning process compared to conventional prompting approaches. Although previous studies have shown the effectiveness of ToT using advanced language models such as GPT-4, further evaluation and reproduction studies remain important for understanding its practical behavior under different settings. In particular, the original study compared beam widths of 1 and 5 using GPT-4, but did not examine an intermediate beam width or report how the token cost of ToT changes with beam width under a weaker model such as GPT-3.5. Motivated by this gap, this work evaluates BFS-based ToT with beam widths of 3 and 5 using GPT-3.5 on the Game of 24 benchmark.

III. Methodology

A. Data Description

This study evaluates the reasoning performance of different prompting and search-based reasoning strategies using the *Game of 24* benchmark. The Game of 24 is a mathematical reasoning task in which a model is given four integers and must generate a valid arithmetic expression that evaluates exactly to 24. The allowed arithmetic operations include addition, subtraction, multiplication, and division, and each number must be used exactly once.

The Game of 24 benchmark is widely used for evaluating multi-step reasoning capabilities because it requires planning, intermediate decision making, arithmetic consistency, and exploration of intermediate solution states. Unlike simple question-answering tasks, solving Game of 24 problems often requires the model to test multiple candidate reasoning paths before obtaining a correct solution. Therefore, the benchmark is highly suitable for evaluating structured reasoning frameworks such as ToT.

In this work, the dataset provided in the original ToT framework was used for all experiments. The dataset ranks 1,362 games by average human solving time, and the original study used games ranked 901–1,000, which are relatively hard, as its test set. To reduce computational cost, we evaluated the first 30 of these test games (ranks 901–930), and all methods were evaluated on the same 30 problems.

Each sample in the dataset contains four integer values ranging from 1 to 13. The objective is to construct a mathematically valid expression equal to 24 using all four numbers exactly once. The generated expression is automatically verified for correctness using arithmetic evaluation rules.

The benchmark enables objective evaluation because each generated solution can be automatically verified according to arithmetic correctness. This property makes the dataset particularly suitable for comparing reasoning strategies using quantitative performance metrics.

Because these games belong to the harder part of the benchmark, they provide a demanding test of whether tree-based search can recover from early reasoning errors.

B. Evaluation Metrics

To evaluate the effectiveness and computational characteristics of different prompting and reasoning strategies, this study uses two primary evaluation metrics: *accuracy* and *token usage*. These metrics enable quantitative comparison between conventional prompting methods and search-based reasoning frameworks such as ToT.

The selected metrics are designed to measure both reasoning performance and computational overhead. Since ToT performs multiple reasoning expansions and intermediate state evaluations, analyzing token consumption is essential for understanding the trade-off between reasoning quality and computational cost.

1) Accuracy

Accuracy is the primary metric used to evaluate reasoning performance on the Game of 24 benchmark. A generated solution is considered correct if:

- The final arithmetic expression evaluates exactly to 24.
- All input numbers are used exactly once.
- Only valid arithmetic operations are applied.

Following the implementation in the original ToT framework, each generated output is assigned a reward value:

$$r = \begin{cases} 1, & \text{if } \mathrm{nums}(expr) = \mathrm{nums}(input) \\ & \text{and } \mathrm{simplify}(expr) = 24 \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

where r represents the correctness reward for a generated solution, and $\mathrm{nums}(\cdot)$ denotes the multiset of numbers appearing in the expression or the input, so an output is rewarded only if it uses exactly the four given numbers and evaluates to 24.

For each problem, the framework may generate multiple candidate outputs. A problem is considered successfully solved if at least one generated output is correct. Therefore, the final accuracy is computed using the following equation:

$$Accuracy = \frac{cnt_{any}}{n} \tag{2}$$

where:

- cnt_{any} denotes the number of problems with at least one correct output.
- n represents the total number of problems.

For IO and CoT, one output is sampled per problem, so this metric equals the per-output success rate. For ToT, all b candidates retained at the final step are checked, and a problem counts as solved if any of them is correct. Because this favours configurations that return more candidates, we additionally report the average success rate over all final candidates (cnt_{avg}/n) for ToT.

This metric directly reflects the reasoning capability of each prompting and search strategy on mathematical reasoning tasks.

2) Token Usage

Token usage measures the total number of tokens consumed during the reasoning process. In the ToT framework, token consumption includes:

- Prompt tokens: input tokens sent to the language model.
- Completion tokens: output tokens generated by the language model.

Since ToT explores multiple reasoning branches and evaluates intermediate thoughts, it generally consumes more tokens than conventional prompting approaches such as IO prompting and CoT prompting.

The total computational cost is computed based on the token usage of both prompts and generated completions. The general cost calculation formula is defined as follows:

$$Cost = \frac{C_t \times P_c + P_t \times P_p}{1000} \tag{3}$$

where:

- C_t denotes the total number of completion (output) tokens.
- P_t denotes the total number of prompt (input) tokens.
- P_c represents the price per 1,000 completion tokens.
- P_p represents the price per 1,000 prompt tokens.

Following the original implementation, we use $P_p = \$0.0015$ and $P_c = \$0.002$ per 1,000 tokens for GPT-3.5-turbo.

Token usage serves as an important indicator of computational complexity and API cost. Higher token consumption generally indicates more extensive reasoning exploration and larger search overhead.

The combination of accuracy and token usage enables comprehensive analysis of both reasoning effectiveness and computational trade-offs across different prompting and search-based reasoning methods.

C. Prompting and Reasoning Methods

This study compares multiple prompting and reasoning strategies for solving mathematical reasoning tasks on the Game of 24 benchmark. The evaluated methods include conventional IO prompting, CoT prompting, and ToT reasoning using BFS with beam widths of 3 and 5.

The objective of this comparison is to analyze how structured reasoning and search-based exploration affect reasoning accuracy and computational cost in LLMs.

1) IO Prompting

IO prompting is the simplest prompting strategy evaluated in this study and serves as a baseline method for comparison with more advanced reasoning frameworks. In the original ToT paper [4], IO prompting was used as a direct-answer baseline in which the language model generates a final solution immediately from the input without producing intermediate reasoning steps.

For the Game of 24 task, the model receives four input numbers and is instructed to generate an arithmetic expression equal to 24 using the given numbers exactly once. Unlike structured reasoning approaches, IO prompting does not perform explicit planning, intermediate state evaluation, branch exploration, or backtracking.

Following the original implementation, the IO prompt is a five-shot prompt. It states the task (use the given numbers and basic arithmetic operations +, $-$, $\times$, $\div$ to obtain 24) and lists five input–answer pairs, each consisting of four numbers and a single expression equal to 24, before presenting the test input. The model therefore outputs one expression directly.

The IO prompting strategy directly maps the input problem to a final output expression in a single generation step. As discussed in the original ToT framework, this approach is computationally efficient because it requires only one reasoning trajectory and minimal search overhead. However, the lack of intermediate reasoning and self-correction mechanisms makes IO prompting less effective for complex multi-step arithmetic reasoning tasks.

Despite its limitations, IO prompting provides an important baseline for evaluating the effectiveness of more advanced reasoning methods such as CoT and ToT.

2) CoT Prompting

CoT prompting is a reasoning strategy that encourages LLMs to generate intermediate reasoning steps before producing a final answer. Unlike IO prompting, which directly generates a solution, CoT decomposes complex problems into sequential reasoning processes that resemble human step-by-step problem solving.

For the Game of 24 task, the model is instructed to explicitly generate intermediate arithmetic operations and partial calculations before reaching the final expression equal to 24. This reasoning decomposition helps the model maintain arithmetic consistency and improves its ability to solve multi-step mathematical reasoning tasks.

The CoT prompt, also taken from the original implementation, is a five-shot prompt that adds the rule that each step may combine only two of the remaining numbers into a new number. Each example shows three intermediate steps in the form `4 + 8 = 12 (left: 4 6 12)`, followed by the final expression, so the model is guided to write the same three-step trajectory before giving its answer.

Compared to IO prompting, CoT prompting introduces explicit intermediate reasoning but still follows a fundamentally linear reasoning structure. Once the model commits to an incorrect reasoning path, it cannot backtrack or explore alternative candidate solutions. Consequently, arithmetic errors occurring in early reasoning steps may propagate throughout the reasoning process and lead to incorrect final outputs.

Nevertheless, CoT prompting remains an important baseline reasoning strategy because it introduces explicit intermediate reasoning and serves as the conceptual foundation for more advanced structured reasoning frameworks such as ToT.

3) ToT Prompting

ToT is a structured reasoning framework proposed to improve the reasoning capability of LLMs through search-based exploration. Unlike conventional prompting approaches such as IO and CoT, ToT organizes intermediate reasoning steps into a tree structure that enables the model to explore multiple candidate solution paths before generating a final answer.

For the Game of 24 task, the ToT framework allows the language model to iteratively generate intermediate arithmetic states, evaluate candidate thoughts, and select promising branches for further exploration. This reasoning process is inspired by classical search algorithms in artificial intelligence, where intermediate states are explored systematically rather than following a single linear reasoning trajectory.

In the original ToT framework, each reasoning state corresponds to a partial solution, while candidate thoughts represent possible next reasoning steps. The framework performs repeated cycles of:

- Thought generation
- State evaluation
- Branch selection
- Search expansion

This structured reasoning mechanism enables the model to revisit alternative reasoning paths and potentially recover from incorrect intermediate decisions.

ToT uses two prompts from the original implementation. A one-shot *propose* prompt asks the model to list possible next steps from the current remaining numbers (for example, `14 - 8 = 6 (left: 2 6 8)`), and each proposal becomes a candidate state. A *value* prompt then asks the model whether the remaining numbers can still reach 24, answering *sure*, *likely*, or *impossible*; these judgements are converted into a numeric score for each state. After three arithmetic steps, the final expression is written with the CoT prompt conditioned on the selected steps and judged with a separate last-step value prompt.

D. Experimental Setup

All experiments used the `gpt-3.5-turbo` model through the OpenAI API with a temperature of 0.7, together with the official ToT implementation [6] and its default prompts. IO and CoT generate one output per problem. ToT runs four search steps (three arithmetic steps and a final answer step). At each step it generates candidate states with the propose prompt, scores each candidate with three samples of the value prompt, and greedily keeps the b highest-valued states, with $b \in \{3, 5\}$. All configurations were evaluated on the same 30 problems. The code, run logs, and plotting scripts are publicly available (see Code Availability).

IV. Results

A. Comparison with the Original ToT Paper

This subsection compares the reproduction results of each method with those reported in the original ToT paper [4]. The original IO and CoT accuracies are averaged over 100 samples per problem, whereas this study draws one sample per problem; both therefore measure the success rate of a single output.

1) IO Prompting

According to the original ToT paper, IO prompting achieved an accuracy of 7.3% on the Game of 24 benchmark using GPT-4 on 100 problems. In this study, the IO prompting strategy was reproduced using GPT-3.5 on 30 problems and achieved an accuracy of 3.3%.

Table 1 presents the comparison between the original implementation and the reproduction results obtained in this study.

Table 1: Comparison of IO Prompting Results

Study	Model	Problems	Accuracy
Original Paper	GPT-4	100	7.3%
Original Paper	GPT-3.5	100	6.0%
This Study	GPT-3.5	30	3.3%

The reproduction result is lower than both values reported in the original paper, although with 30 problems the difference from the GPT-3.5 result corresponds to roughly one problem. An inspection of the run log shows that in 28 of the 29 failed problems, GPT-3.5 used the four given numbers but produced an expression that does not evaluate to 24 while still claiming that it does; in the remaining failure, the expression used numbers that were not given.

2) CoT Prompting

According to the original ToT paper, CoT prompting achieved an accuracy of 4.0% using GPT-4 on 100 problems. The paper also reported an accuracy of 3.0% using GPT-3.5 under the same benchmark setting. In this study, the CoT prompting strategy was reproduced using GPT-3.5 on 30 problems and achieved an accuracy of 3.3%.

Table 2 presents the comparison between the original experimental results and the reproduction results obtained in this study.

Table 2: Comparison of CoT Prompting Results

Study	Model	Problems	Accuracy
Original Paper	GPT-4	100	4.0%
Original Paper	GPT-3.5	100	3.0%
This Study	GPT-3.5	30	3.3%

The reproduction results obtained in this study are generally consistent with the GPT-3.5 performance reported in the original paper. Although CoT prompting improves reasoning transparency by exposing intermediate thought processes, the overall reasoning performance remains relatively limited for GPT-3.5 on the Game of 24 benchmark.

An inspection of the 29 failed CoT outputs shows that calculation errors were not the main cause of failure. Only 4 outputs contained an arithmetic error in an intermediate step. In 17 cases, the final expression used the four given numbers but did not evaluate to 24; in 7 cases it used numbers that were not given; and in 1 case no final answer was produced. This suggests that GPT-3.5 usually committed early to a sequence of operations that could not reach 24, and the linear CoT structure provided no way to recover from this choice.

3) ToT

In the original ToT paper, the ToT framework demonstrated substantial improvements over conventional prompting strategies on the Game of 24 benchmark. Using GPT-4 with beam width = 1, the framework achieved an accuracy of 45% on 100 problems. Increasing the beam width to 5 further improved the accuracy to 74%, demonstrating the effectiveness of broader search exploration.

The original paper also evaluated GPT-3.5 using beam width = 5 and reported an accuracy of 19% on 100 problems. This result indicates that structured search exploration can substantially improve reasoning performance even for weaker language models.

In this study, the ToT framework was reproduced using GPT-3.5 on 30 problems with beam widths of 3 and 5. The reproduction experiments achieved an accuracy of 10% with beam width = 3 and 30% with beam width = 5. A beam width of 1 was only run as a single-problem pilot due to API budget constraints and is therefore excluded from the quantitative comparison.

Table 3 presents the comparison between the original experimental results and the reproduction results obtained in this study.

Table 3: Comparison of ToT Results

Study	Model	Beam Width	Problems	Accuracy
Original Paper	GPT-4	1	100	45%
Original Paper	GPT-4	5	100	74%
Original Paper	GPT-3.5	5	100	19%
This Study	GPT-3.5	3	30	10%
This Study	GPT-3.5	5	30	30%

The experimental results obtained in this study reveal several important observations regarding the behavior of search-based reasoning under GPT-3.5.

First, increasing the beam width from 3 to 5 improved reasoning accuracy from 10% (3 of 30 problems) to 30% (9 of 30 problems).

These observations suggest that broader search exploration substantially increases the probability of discovering valid arithmetic solution paths. By retaining more candidate branches during the reasoning process, the framework gains additional opportunities to recover from incorrect intermediate reasoning states through alternative candidate solutions.

Interestingly, the reproduction result obtained with beam width = 5 achieved 30% accuracy, which is higher than the 19% accuracy reported for GPT-3.5 in the original paper. However, with only 30 problems the 95% confidence interval for this estimate is wide (approximately 17%–48%, Wilson score interval [7]), so the difference from the reported 19% should not be interpreted as a genuine improvement over the original result.

The results also demonstrate the strong relationship between search breadth and reasoning capability. Compared to IO and CoT prompting, ToT with larger beam widths achieved substantially better reasoning performance because the framework was able to evaluate multiple reasoning trajectories instead of relying on a single linear reasoning chain.

Nevertheless, the reproduction results remained substantially lower than the GPT-4 performance reported in the original paper. GPT-4 achieved 74% accuracy using beam width = 5, whereas GPT-3.5 in this study achieved 30% accuracy under the same beam width configuration. This observation suggests that the effectiveness of ToT strongly depends not only on the search strategy but also on the reasoning capability of the underlying language model.

A likely contributing factor is that ToT relies on the same model both to propose next steps and to evaluate intermediate states, so weaker proposals and less reliable value judgements from GPT-3.5 reduce the benefit of the search compared to GPT-4.

Overall, the experimental results support the theoretical motivation of the ToT framework: broader search exploration substantially improves reasoning performance by enabling the model to explore and evaluate multiple candidate solution paths before generating the final answer.

B. Main Performance Comparison

This subsection presents the overall comparison between conventional prompting strategies and search-based reasoning frameworks on the Game of 24 benchmark. The evaluation focuses on reasoning accuracy, token consumption, and computational cost under different prompting and reasoning configurations.

Table 4 summarizes the main experimental results obtained in this study.

Table 4: Main Performance Comparison

Method	Accuracy	Completion Tokens	Prompt Tokens	Cost ($)
IO	3.3%	526	5400	0.009152
CoT	3.3%	3063	13110	0.025791
ToT ($b = 3$)	10%	129429	533440	1.059018
ToT ($b = 5$)	30%	182851	728533	1.458502

The experimental results demonstrate substantial differences in both reasoning capability and computational efficiency across prompting and reasoning strategies. In general, methods with broader reasoning exploration achieved higher accuracy but required significantly larger computational resources.

1) IO vs CoT

The comparison between IO and CoT prompting reveals that explicit step-by-step reasoning did not improve reasoning accuracy for GPT-3.5 on the evaluated Game of 24 problems. Both methods achieved the same accuracy of 3.3%.

However, CoT prompting required substantially more computational resources than IO prompting. Specifically, CoT consumed 3063 completion tokens and 13110 prompt tokens, compared to only 526 completion tokens and 5400 prompt tokens for IO prompting. Consequently, the estimated API cost of CoT prompting was approximately three times higher than that of IO prompting.

These observations suggest that although CoT introduces explicit reasoning decomposition, the intermediate steps generated by GPT-3.5 frequently led to states from which 24 could not be reached. Since CoT follows a linear reasoning structure without branch exploration or backtracking, such early choices could not be revised.

To provide a clearer comparison between direct prompting and linear reasoning strategies, Table 5 summarizes the differences between IO and CoT prompting.

Table 5: IO vs CoT Comparison

Metric	IO	CoT
Accuracy	3.3%	3.3%
Completion Tokens	526	3063
Prompt Tokens	5400	13110
Estimated Cost ($)	0.009152	0.025791
Reasoning Style	Direct Answer	Step-by-step Reasoning
Search Capability	None	None

Although CoT generated more detailed reasoning processes, the additional computational cost did not translate into measurable improvements in reasoning accuracy under GPT-3.5.

2) CoT vs ToT

The comparison between CoT and ToT demonstrates the impact of structured search-based reasoning on mathematical problem solving.

Unlike CoT, which follows a single reasoning trajectory, ToT explores multiple candidate reasoning branches through iterative thought generation and state evaluation. This structured exploration enables the framework to revisit alternative reasoning paths and recover from incorrect intermediate reasoning states.

The experimental results indicate that broader search exploration substantially improved reasoning performance for GPT-3.5. While CoT achieved only 3.3% accuracy, ToT with beam width = 5 achieved 30% accuracy, representing a significant improvement in reasoning capability.

However, the performance improvement introduced extremely large computational overhead. ToT with beam width = 5 consumed 182851 completion tokens and 728533 prompt tokens, resulting in an estimated API cost of approximately $1.46. In contrast, CoT consumed only 3063 completion tokens and 13110 prompt tokens with an estimated cost of approximately $0.026.

These observations reveal an important trade-off between reasoning performance and computational efficiency in search-based reasoning frameworks.

To further illustrate the differences between linear reasoning and structured search exploration, Table 6 compares CoT prompting with the best-performing ToT configuration.

Table 6: CoT vs ToT Comparison

Metric	**CoT**	**ToT** ($b = 5$)
Accuracy	3.3%	30%
Completion Tokens	3063	182851
Prompt Tokens	13110	728533
Estimated Cost ($)	0.025791	1.458502
Reasoning Structure	Linear	Tree-based Search
Backtracking	No	Yes
Branch Exploration	Single Path	Multiple Candidate Paths

The substantial increase in token consumption highlights one of the major limitations of search-based reasoning frameworks. Although broader search exploration can significantly improve reasoning accuracy, it also dramatically increases computational complexity and API cost.

Overall, the results suggest that ToT can substantially improve reasoning performance compared to conventional prompting strategies, but its effectiveness depends heavily on computational resources and search breadth.

C. Overall Performance Analysis

Figure 1 presents the accuracy comparison between IO, CoT, and Tree of Thoughts with beam widths of 3 and 5 on the Game of 24 benchmark. The experimental results reveal substantial differences in reasoning performance across prompting and search-based reasoning strategies. In particular, the comparison highlights how search exploration influences the reasoning capability of large language models when solving arithmetic reasoning tasks.

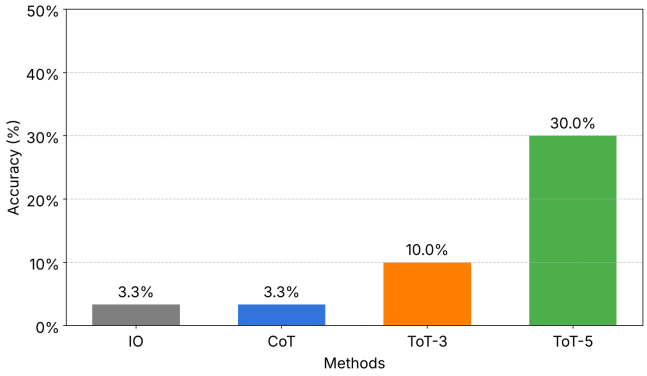


Fig. 1: Problem-level accuracy comparison: IO, CoT, ToT-3, ToT-5

Both IO and CoT achieved the same accuracy of 3.3%, indicating that linear step-by-step reasoning did not significantly improve mathematical reasoning performance under GPT-3.5. This observation suggests that although CoT generates explicit intermediate reasoning steps, the model often commits early to a sequence of operations that cannot reach 24 and has no mechanism to revise it.

In contrast, the ToT framework achieved noticeably higher accuracy. With beam width = 3, ToT solved 3 of 30 problems (10%), and with beam width = 5 it solved 9 of 30 problems (30%), the best overall performance among the evaluated methods. Measured as the average success rate over all final candidates rather than per problem, the corresponding figures are 3.3% and 9.3%, respectively.

These results suggest that broader search exploration substantially improves reasoning capability by enabling the model to evaluate multiple candidate reasoning paths rather than relying on a single linear reasoning trajectory.

Overall, the experimental findings support the theoretical motivation of the ToT framework, where structured search exploration can improve reasoning performance through branch exploration and intermediate state evaluation. Nevertheless, the relatively low overall accuracy compared to GPT-4 results reported in the original paper indicates that the effectiveness of ToT strongly depends on the reasoning capability of the underlying language model.

D. Computational Cost Analysis

Figure 2 presents the computational cost comparison between IO, CoT, and Tree of Thoughts with beam width = 5 on the Game of 24 benchmark using a logarithmic scale. The figure highlights substantial differences in API usage and computational expense across prompting-based and search-based reasoning strategies. In particular, the comparison demonstrates how increasing reasoning complexity and search exploration significantly affect token consumption and overall inference cost when solving arithmetic reasoning tasks with large language models.

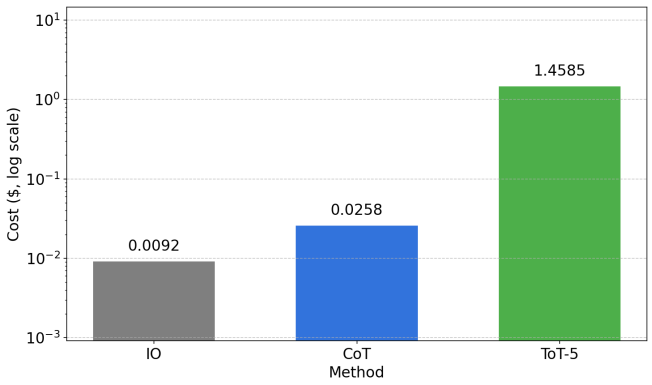


Fig. 2: Computational cost comparison between IO, CoT, and ToT ($b = 5$) using logarithmic scale.

Among the evaluated methods, IO achieved the lowest computational cost with an estimated API expense of approximately \$0.0092. This result is expected because IO directly generates the final answer without producing extensive intermediate reasoning steps or performing search-based exploration.

CoT prompting increased the computational cost to approximately \$0.0258, which is nearly three times higher than IO prompting. The increase in cost is mainly caused by the generation of detailed step-by-step reasoning chains that require additional completion tokens and longer prompts.

In contrast, the ToT framework with beam width = 5 introduced extremely large computational overhead, reaching approximately \$1.4585. Compared to CoT prompting, the ToT framework required substantially more token usage because multiple candidate reasoning branches were generated, evaluated, and expanded during the search process. The logarithmic scale in Figure 2 clearly illustrates the dramatic increase in computational cost introduced by broader search exploration.

Despite the substantial increase in API cost, ToT achieved significantly higher reasoning accuracy compared to IO and CoT. These findings demonstrate an important trade-off between reasoning performance and computational efficiency in search-based reasoning frameworks. While broader search exploration can substantially improve reasoning capability, it also dramatically increases token consumption and computational expense.

Overall, the experimental results suggest that ToT provides improved reasoning performance at the cost of significantly higher computational complexity, highlighting the importance of balancing reasoning accuracy and resource efficiency when deploying search-based reasoning systems.

E. Impact of Beam Width on ToT Performance

Beam width is one of the most important hyperparameters in the ToT framework because it determines the number of candidate reasoning branches retained during each search iteration. A larger beam width allows broader exploration of candidate solution paths, while a smaller beam width restricts the search space and reduces computational cost.

Figure 3 illustrates the relationship between beam width and reasoning accuracy for the ToT framework on the Game of 24 benchmark. Within the evaluated range, a larger beam width led to higher accuracy.

The experimental results show a clear difference between the two evaluated beam widths:

- Beam width = 3 achieved 10% accuracy (3 of 30 problems)
- Beam width = 5 achieved 30% accuracy (9 of 30 problems)

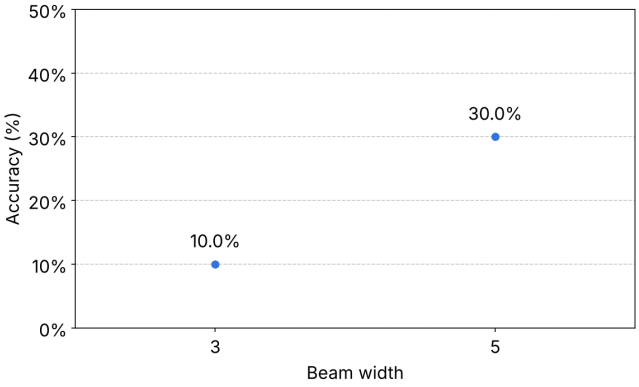


Fig. 3: Beam width versus accuracy for ToT on the Game of 24 benchmark.

With beam width = 3, the framework retained three candidate states at each search step, which allowed it to solve three times as many problems as IO and CoT. Increasing the beam width to 5 tripled the number of solved problems again, as the broader search space provided more opportunities to recover from incorrect intermediate reasoning states.

This improvement came at a moderate additional cost. Compared to beam width = 3, the beam width = 5 configuration increased:

- Completion tokens from 129,429 to 182,851 (1.41×)
- Prompt tokens from 533,440 to 728,533 (1.37×)
- Estimated API cost from \$1.059 to \$1.459 (1.38×)

Between these two settings, the cost grew by less than the ratio of beam widths ($5/3 \approx 1.67\times$), while the number of solved problems tripled. However, both ToT configurations were far more expensive than the prompting baselines: beam width = 3 cost about 41 times as much as CoT, and beam width = 5 about 57 times.

These observations indicate that, within the evaluated range, increasing search breadth improves reasoning accuracy. Because only two beam widths were evaluated on 30 problems, this result should be read as indicative rather than as a general scaling trend.

F. Accuracy-Cost Trade-off Analysis

Figure 4 illustrates the relationship between reasoning accuracy and computational cost across the four evaluated reasoning strategies: IO, CoT, and ToT with beam widths of 3 and 5. The horizontal axis represents the estimated API

cost in logarithmic scale, while the vertical axis represents the reasoning accuracy achieved on the Game of 24 benchmark.

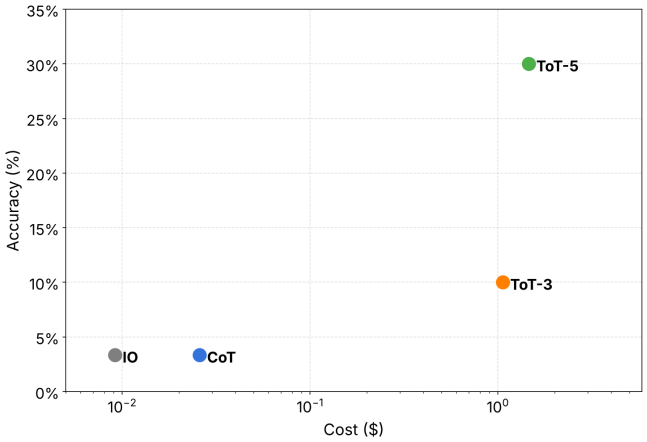


Fig. 4: Relationship between reasoning accuracy and computational cost across IO, CoT, and ToT configurations.

The results reveal a clear trade-off between reasoning performance and computational expense. IO prompting achieved the lowest computational cost at approximately \$0.009 while maintaining an accuracy of 3.3%. CoT prompting increased the computational cost to approximately \$0.026 without improving reasoning accuracy, suggesting that linear step-by-step reasoning alone was insufficient to enhance arithmetic reasoning performance under GPT-3.5.

In contrast, the ToT framework achieved substantially higher reasoning accuracy, but these improvements were accompanied by dramatically higher computational costs. ToT with beam width = 3 reached 10% accuracy at an estimated cost of approximately \$1.06, while beam width = 5 achieved the highest reasoning accuracy of 30% at approximately \$1.46.

The logarithmic cost axis in Figure 4 shows that ToT with $b = 3$ and $b = 5$ costs between $41\times$ and $159\times$ more than IO and CoT, while the increase from $b = 3$ to $b = 5$ is comparatively modest (about $1.4\times$). Although ToT substantially improved reasoning capability compared to IO and CoT, the framework required significantly larger token consumption due to branch generation, intermediate state evaluation, and search expansion.

These findings indicate that search-based reasoning frameworks introduce an important balance between reasoning effectiveness and computational efficiency. While broader search exploration can substantially improve reasoning accuracy, the associated computational overhead may limit scalability in practical deployments.

Overall, the experimental results demonstrate that ToT provides the strongest reasoning performance among the evaluated methods, but at the cost of significantly increased API usage and computational complexity.

V. Conclusion and Discussion

A. Conclusion

This paper presented a comparative evaluation of IO, CoT, and ToT reasoning frameworks on the Game of 24 mathematical reasoning benchmark using GPT-3.5. The experimental results demonstrated substantial differences in reasoning performance and computational efficiency across the evaluated methods.

The findings showed that conventional prompting approaches, including IO and CoT, achieved relatively low reasoning accuracy, both obtaining 3.3% accuracy on the evaluation dataset. Although CoT introduced explicit step-by-step reasoning, it did not significantly improve arithmetic reasoning performance under GPT-3.5, suggesting that linear reasoning alone is insufficient for solving complex mathematical reasoning tasks.

In contrast, the ToT framework demonstrated noticeable performance improvements through structured search exploration. The experimental results showed that increasing the beam width from 3 to 5 improved reasoning accuracy from 10% to 30%. These findings support the theoretical motivation of ToT, where exploring multiple candidate reasoning branches enables more effective problem-solving compared to single-path reasoning strategies.

However, the improved reasoning performance was accompanied by dramatically higher computational cost. The ToT framework required significantly larger token consumption and API expense due to branch generation, intermediate state evaluation, and search expansion. The experimental analysis therefore revealed an important trade-off between reasoning effectiveness and computational efficiency in search-based reasoning systems.

Furthermore, the comparison with results reported in the original ToT paper highlighted the strong dependence of reasoning performance on the capability of the underlying language model. While GPT-4 achieved substantially higher accuracy in the original study, GPT-3.5 demonstrated more limited reasoning capability even under broader search exploration settings.

Overall, this study suggests that structured search-based reasoning frameworks can substantially improve mathematical reasoning performance compared to conventional prompting approaches. Nevertheless, the effectiveness of ToT remains strongly influenced by model capability and computational resource requirements.

This study is limited to 30 problems and two beam widths, and the $b = 1$ configuration was only run as a single-problem pilot. Future work may extend this study by evaluating all beam widths on the full 100-problem test set, evaluating ToT on larger mathematical reasoning benchmarks such as GSM8K or SVAMP, exploring alternative search strategies, and comparing open-source language models with commercial LLMs in search-based reasoning tasks.

B. Insights and Discussion

The experimental results provide several important insights into the behavior of search-based reasoning frameworks for mathematical problem solving.

First, the findings indicate that simply generating step-by-step reasoning does not necessarily improve reasoning accuracy. Although CoT prompting encourages the model to produce intermediate reasoning steps, the accuracy remained identical to IO prompting at 3.3%. The failed CoT outputs indicate that the main problem was not calculation errors but early choices of operations that could not lead to 24, which a single linear reasoning chain cannot revise. Therefore, linear reasoning alone may be insufficient for solving complex mathematical reasoning tasks.

Second, the results demonstrate that structured search exploration plays a critical role in improving reasoning performance. Unlike IO and CoT, the ToT framework allows the model to generate and evaluate multiple candidate reasoning branches during problem solving. The experiments showed that increasing the beam width from 3 to 5 improved reasoning accuracy from 10% to 30%. This finding supports the hypothesis that broader search exploration increases the probability of discovering valid reasoning paths while reducing the impact of early reasoning mistakes.

Third, the study highlights the strong relationship between beam width and computational complexity. Although larger beam widths improved reasoning accuracy, they also introduced dramatically higher computational cost. The ToT configuration with beam width = 5 achieved the best reasoning performance but required more than $1.45 in estimated API cost, which was substantially larger than both IO and CoT prompting. This demonstrates that search-based reasoning frameworks introduce a significant trade-off between reasoning effectiveness and computational efficiency.

Another important observation is that the effectiveness of ToT strongly depends on the capability of the underlying language model. Compared to the original paper, which reported substantially higher performance using GPT-4, the GPT-3.5 experiments in this study achieved lower reasoning accuracy across all prompting methods. This suggests that search-based reasoning frameworks cannot fully compensate for the reasoning limitations of weaker language models. Instead, the reasoning quality of the base model directly influences the effectiveness of search exploration and branch evaluation.

Overall, the experimental findings suggest that search-based reasoning frameworks represent a promising direction for improving mathematical reasoning in LLMs. However, practical deployment of such frameworks must carefully balance reasoning performance, computational efficiency, and model capability to achieve scalable and cost-effective reasoning systems.

Code Availability

The code, prompts, run logs, and plotting scripts used in this study are available at https://github.com/HaiTriu301/tree-of-thought. The repository is a fork of the official ToT implementation [6]; the search code, tasks, and prompts are unchanged, and the GPT-3.5 run logs and plotting scripts were added for this study.

Acknowledgment

The authors would like to thank the course instructor for the guidance and feedback provided throughout this project, and our classmates for their support.

References

[1] T. Brown *et al.*, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[2] OpenAI, "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[3] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.

[4] S. Yao *et al.*, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 11809–11822.

[5] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[6] S. Yao *et al.*, "Official repository of Tree of Thoughts (ToT)," GitHub, 2023. [Online]. Available: https://github.com/princeton-nlp/tree-of-thought-llm

[7] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.